Java 例外処理～実行時のトラブルを読み解き、扱う～

1. 本単元の概要

プログラムは**コンパイルが通っても、実行中に想定外のことが起きる**ことがあります。Java ではそのとき投げられるのが**例外（Exception）**です。

この単元では、**例外の見方**と**try / catch / finally**、**複数の catch の順序**、**try-with-resources** までを、短いコードでたどります。

2. 学習目標

次の項目を、図やコードとあわせて説明できることを目指します。

▼ コンパイルエラーと実行時のトラブルを区別できる

コンパイルは「文法と型のチェック」、実行中の例外は「動いているときに起きた異常」として切り分けられる。

▼ チェック例外と非チェック例外をざっくり説明できる

チェック例外は呼び出し側に処理を強いることが多い。非チェック例外（RuntimeException 系）は実行時に起きやすい論理ミス・前提崩れとして扱われる。

▼ try / catch / finally の流れを追える

try で囲んだ範囲で例外が起きると、対応する catch にジャンプする。finally は（ほぼ）必ず通る。

▼ 複数 catch で広い型（Exception）を最後に置く理由を言える

上から順に「この型か？」と照合するため、親クラスを先に書くと子の catch に届かない。

▼ multi-catch（|）の役割を言える

同じ処理で複数種類の例外をまとめて捕まえたいときに使う（catch ブロックの重複を減らす）。

▼ try-with-resources が何のためか一言で言える

開いたリソース（close が必要なもの）を、ブロックの終わりで確実に閉じるための書き方。

3. 本編

3-1. コンパイルエラーと実行時の違い

- **コンパイルエラー**：実行前の段階でコンパイラが見つける誤り（型の不一致、未定義の名前など）。プログラムは**起動できない**。
- **実行時の例外**：起動後の処理の途中で起きる。例：0 で割る、配列の範囲外、想定外の入力値など。

3-2. Throwable の階層（ざっくり）

`Throwable` の下に `Error` と `Exception` があります。入門では **Error は捕まえない**（VM や環境側の深刻な問題が多い）方針でよいです。

`Exception` のうち `RuntimeException` とそのサブクラスが**非チェック例外**、それ以外の多くが**チェック例外**です。

3-3. 自販機の例：非チェック例外（IllegalArgumentException）

本数が 0 以下なら、メソッドの前提が壊れているので `IllegalArgumentException` を投げます（呼び出し側は try-catch しなくてもコンパイルは通ります＝非チェック）。

```
public class VendingMachine {

        // フィールド
        String drinkName = ”水”;
        int stock = 3;

        // 引数なしのコンストラクタ（中身が空なので1行でよい）
        public VendingMachine() {}

        // 指定本数を購入する。本数が不正なら非チェック例外を投げる
        public void buy(int bottles) {
                if (bottles <= 0) {
                        throw new IllegalArgumentException(”本数は1以上を指定してくださ
                }
                if (stock < bottles) {
                        throw new IllegalArgumentException(”在庫が足りません”);
                }
                stock = stock - bottles;
                System.out.println(drinkName + ”を” + bottles + ”本購入しました。”);
        }
}
```

▼ **上記クラスをドラッグでコピー（クリックで開く）**

下のプレーンテキストの貼り付け欄から、IDE へドラッグで取り込めます。

マウスで全選択して貼り付け欄へドラッグしてください：

```
public class VendingMachine {

        // フィールド
        String drinkName = "水";
        int stock = 3;

        // 引数なしのコンストラクタ（中身が空なので1行でよい）
        public VendingMachine() {}
```

3-4. チェック例外：在庫不足を「必ず処理させる」

独自のチェック例外 `OutOfStockException` を定義し、 `dispense` から `throws` します。呼び出す側は **try-catch か throws** のどちらかが必要です。

```
// チェック例外（Exception のサブクラス）
public class OutOfStockException extends Exception {

        public OutOfStockException(String message) {
                super(message);
        }
}

public class VendingMachineStock {

        int stock = 2;

        public VendingMachineStock() {}

        // 在庫を減らす。足りなければチェック例外を投げる（呼び出し側に処理を強いる）
        public void dispense(int bottles) throws OutOfStockException {
                if (bottles <= 0) {
                        throw new IllegalArgumentException("本数は1以上を指定してくださ
                }
                if (stock < bottles) {
                        String message = "在庫不足（必要:" + bottles + " 実在庫:" + stoc
                        OutOfStockException ex = new OutOfStockException(message);
                        throw ex;
                }
                stock = stock - bottles;
                System.out.println("販売OK。残り在庫=" + stock);
        }
}
```

▼ **OutOfStockException と VendingMachineStock をドラッグでコピー（クリックで開く）**

マウスで全選択して貼り付け欄へドラッグしてください：

```
public class OutOfStockException extends Exception {

        public OutOfStockException(String message) {
                super(message);
        }
}

public class VendingMachineStock {
```

3-5. try / catch / finally（main から呼ぶ）

参照変数は `vm` でそろえると読みやすい、というルールに合わせています。

```
public class VendingMachineDemo {

        public static void main(String[] args) {
                VendingMachine vm = new VendingMachine();
                try {
                        // 意図的に在庫より多く買おうとする例（非チェック）
                        vm.buy(10);
                } catch (IllegalArgumentException e) {
                        System.out.println("購入できません：" + e.getMessage());
                } finally {
                        System.out.println("処理ブロックを抜けます（finally）");
                }

                VendingMachineStock vmStock = new VendingMachineStock();
                try {
                        vmStock.dispense(1);
                        vmStock.dispense(5);
                } catch (OutOfStockException e) {
                        System.out.println("チェック例外を捕捉：" + e.getMessage());
                }
        }
}
```

▼ **VendingMachineDemo をドラッグでコピー（クリックで開く）**

マウスで全選択して貼り付け欄へドラッグしてください：

```
public class VendingMachineDemo {

        public static void main(String[] args) {
                VendingMachine vm = new VendingMachine();
                try {
                        vm.buy(10);
                } catch (IllegalArgumentException e) {
                        System.out.println("購入できません：" +
```

3-6. 複数 catch の順序と multi-catch（|）

順序：より具体的な型を上に、`Exception` のような広い型は**最後**。

multi-catch：同じ復旧処理でよいとき、型を `|` で並べます（catch パラメータは事実上 final 扱い）。

```
public class MultiCatchSample {

        public static void main(String[] args) {
                String text = null;
                try {
                        System.out.println(text.length());
                } catch (NullPointerException | IllegalArgumentException e) {
                        System.out.println("引数や参照が不正: " + e.getClass().getSimple
                } catch (Exception e) {
                        System.out.println("その他: " + e.getMessage());
                }
        }
}
```

▼ MultiCatchSample をドラッグでコピー（クリックで開く）

マウスで全選択して貼り付け欄へドラッグしてください：

```
public class MultiCatchSample {

        public static void main(String[] args) {
                String text = null;
                try {
                        System.out.println(text.length());
                } catch (NullPointerException | IllegalArgumentException e)
{
```

3-7. try-with-resources（close を自動で呼ぶ）

`AutoCloseable`（多くは `Closeable`）を実装したオブジェクトを、try の丸括弧の中で宣言すると、ブロックを抜けるときに `close()` が呼ばれます。ファイルや DB 接続など「閉じ忘れがバグになるもの」で威力を発揮します。

```
public class LogResource implements AutoCloseable {

        public void writeLine(String line) {
                System.out.println(”[LOG] ” + line);
        }

        @Override
        public void close() {
                System.out.println(”[LOG] リソースを閉じました”);
        }
}

public class TryWithResourcesSample {

        public static void main(String[] args) {
                try (LogResource log = new LogResource()) {
                        log.writeLine(”処理中”);
                }
        }
}
```

▼ LogResource と TryWithResourcesSample をドラッグでコピー（クリックで開く）

マウスで全選択して貼り付け欄へドラッグしてください：

```
public class LogResource implements AutoCloseable {

        public void writeLine(String line) {
                System.out.println("[LOG] " + line);
        }

        @Override
        public void close() {
```

4. ラボ（10分チェックリスト）

10分で終わる最小確認

1. `VendingMachine#buy` で在庫不足を起こし、catch でメッセージが出ることを確認する。
2. `OutOfStockException` を使う `dispense` を、main から try-catch で捕捉できることを確認する。
3. finally に `System.out.println` を1行入れ、例外の有無で必ず通ることを目で追う。
4. multi-catch のブロックを一時的に入れ替え、`Exception` を先頭にするとコンパイルが通らない（または到達不能になる）ことを確認する。
5. `TryWithResourcesSample` を実行し、`close` の行がブロック終了後に出ることを確認する。

5. まとめ

例外は「実行中に起きた異常」を表すオブジェクトです。**非チェック**は呼び出し側に処理を強制しません。**チェック**はコンパイラが処理の存在をチェックします。

```
try { 危ない処理 }
catch (具体的な型 e) { 復旧やログ }
catch (Exception e) { 最後の網 }
finally { 後始末（ほぼ必ず） }

try (リソース r = ...) { 処理 }  // 終わりで r.close()
```

追加の問いは、同じフォルダの **03-練習問題-Java例外処理.html** で確認してください。